

GENAI STUDY ASSISTANT WITH RAG

Multi-Step Generative AI & Retrieval-Augmented Generation Workflow for Intelligent Learning

STUDENT	MULINTI SUJITH REDDY
REGISTER NO.	99230041076
DEPARTMENT	CSE(AIML)
DATE	06-09-2026
ASSESSMENT	Mini Assessment — GenAI Workflow Design

01 | Concept

Purpose: The GenAI Study Assistant with RAG allows students to upload academic documents such as notes, textbooks, lecture material, lab manuals, and previous question papers. The system retrieves relevant information from these documents and uses a Generative AI model to produce grounded, understandable answers with source references.

Problem: Students often spend significant time searching through multiple PDFs and notes for a small piece of information. A normal chatbot may also generate answers that are not grounded in the student's study material. A Retrieval-Augmented Generation workflow addresses this by retrieving relevant document content before generating the response.

02 | Objective

The workflow uses sequential prompts, document retrieval, embeddings, a vector database, and an LLM to understand student questions, find relevant study material, generate concise explanations, create learning content, and provide source-grounded responses.

03 | Workflow Overview

1. Ingest	2. Retrieve	3. Generate	4. Verify	5. Learn
Documents	Relevant Chunks	LLM Answer	Grounding Check	Quiz / Summary

04 | Workflow Steps

1. Document Ingestion & Chunking: Load PDFs or text documents, extract their content, divide the content into meaningful chunks, and prepare metadata such as document name, page number, subject, and chapter.

2. Question Understanding: Analyze the student's question and identify the subject, topic, intent, and useful search terms for retrieval.

3. RAG Retrieval: Convert the question into an embedding, search the vector database, and retrieve the most relevant document chunks from the student's study material.

4. Answer Generation: Provide the retrieved context to the LLM through a structured prompt and generate a clear answer that stays grounded in the retrieved content.

5. Verification & Learning Support: Check whether the answer is supported by the retrieved context, then optionally generate a summary, flashcards, MCQs, or practice questions from the same source material.

05 | Prompt Workflow

Step 1 — Process Study Documents

Load uploaded PDFs and extract text, headings, page information, and document metadata. Create chunks suitable for embedding and retrieval.

Step 2 — Understand Student Question

Classify the question as explanation, definition, comparison, summary, problem-solving, or quiz generation and identify the relevant academic topic.

Step 3 — Retrieve Relevant Context

Search the vector database using semantic similarity and return the top relevant chunks. Prefer information from the student's uploaded material.

Step 4 — Generate Grounded Answer

Use a prompt template containing the question and retrieved context. Ask the LLM to answer clearly, avoid unsupported claims, and mention the source document and page when available.

Step 5 — Verification & Learning Output

Check that the response is supported by the retrieved context. If requested, transform the same material into a short summary, flashcards, MCQs, or revision questions.

06 | RAG Prompt Chaining Architecture

STAGE	INPUT / CONTEXT	OUTPUT
1	PDF / DOCX / TXT study material	Clean text + metadata + chunks
2	Student question	Topic + intent + search query
3	Question embedding + vector DB	Top relevant document chunks
4	Question + retrieved context + prompt	Grounded GenAI answer
5	Answer + retrieved sources	Verified response + citations
6	Verified context	Summary / MCQs / flashcards

07 | Core RAG Components

- Document Loader — reads academic PDFs and other supported files.
- Text Splitter — converts long documents into searchable chunks.
- Embedding Model — represents questions and chunks as vectors.
- Vector Database — stores embeddings and performs semantic similarity search.
- Retriever — selects the most relevant chunks for each question.
- LLM — generates the final response using the retrieved context.

08 | Demonstration Input

Example Scenario: A student uploads Operating Systems notes and asks: “*Explain deadlock prevention in simple terms and give the key methods mentioned in my notes.*” The RAG system searches the uploaded study material and retrieves the most relevant chunks.

09 | Expected Final Output & Workflow Result

The system identifies the topic as Operating Systems → Deadlocks, retrieves relevant sections from the uploaded notes, generates a simple explanation, lists the methods supported by the retrieved content, and displays the source document/page information where available.

USER QUESTION	Explain deadlock prevention in simple terms.
RETRIEVAL	Top relevant chunks from Operating Systems notes
GENAI OUTPUT	Simple explanation + key methods
GROUNDING	Answer constrained by retrieved context
SOURCE	Document name + page/chapter when metadata is available

10 | Sample Support Execution Plan

PHASE	ACTION FOCUS	OUTPUT / METRIC
Phase 1	Document Ingestion	Files loaded → text and metadata extracted
Phase 2	Chunking & Embeddings	Chunks converted → vector representations
Phase 3	Semantic Retrieval	Question matched → relevant chunks retrieved
Phase 4	Answer Generation	Context + prompt → grounded response
Phase 5	Verification	Answer checked → unsupported claims reduced
Phase 6	Learning Support	Summary / MCQs / flashcards generated

11 | Key Benefits

- ✓ Reduces time spent manually searching through multiple academic documents
- ✓ Produces answers grounded in the student's own study material
- ✓ Supports semantic document search rather than only keyword matching
- ✓ Provides source information to improve transparency and trust
- ✓ Converts study material into summaries, MCQs, flashcards, and revision content
- ✓ Demonstrates practical Generative AI, RAG, embeddings, prompt engineering, and evaluation

12 | Discussion Questions & Answers

1. What is the advantage of using {user_input} instead of hard-coding input?

Templating separates the fixed prompt structure from dynamic student questions and retrieved context. The same workflow can therefore handle many subjects and question types without hard-coding each question.

2. When would .format() be enough?

Python's .format() is sufficient for simple, linear prompt construction, such as inserting a student's question and a small amount of retrieved context into a fixed prompt.

3. When might Jinja2 be useful?

Jinja2 is useful when prompts become structurally dynamic—for example, when different question types require different instructions, when multiple retrieved sources must be listed, or when optional sections such as citations, summaries, and quizzes need to be included conditionally.

4. Why should prompts be versioned?

Prompt versioning creates an audit trail for changes. If a new prompt produces lower-quality or less-grounded answers, the project can compare versions and roll back to a reliable prompt.

5. What is a prompt regression?

A prompt regression occurs when changing a prompt improves one task but unintentionally reduces performance on another task—for example, improving summaries while making question-answering less precise.

6. Why use the same evaluation set when comparing prompt versions?

A fixed evaluation set allows an apples-to-apples comparison. If both the prompts and test questions change, it becomes difficult to determine whether the performance change came from the prompt or from different test data.

7. Why is RAG useful for a study assistant?

RAG supplies the LLM with relevant material from the student's documents before generation. This helps the assistant answer from the available study content instead of relying only on the model's general knowledge.

13 | Suggested Technology Stack

LAYER	TECHNOLOGY OPTIONS
Language	Python
LLM	OpenAI / Gemini / other course-approved LLM
RAG Framework	LangChain or LlamaIndex
Embeddings	Sentence Transformers / API embeddings
Vector Store	FAISS / Chroma
Document Processing	PyMuPDF / PDF loader
Interface	Streamlit
Evaluation	Golden question set + retrieval/answer quality checks